Annahmen

Dieser Dokument dokumentiert Annahmen, die während der Entwicklung getroffen wurden, weil Anforderungen unklar oder nicht spezifiziert waren.

- Manche Annahmen wurden durch Rückfragen beim Stakeholder konkretisiert.
- Andere Annahmen ergeben sich als Folge von bewussten Architekturentscheidungen und wollten wir zur Transparenz festhalten.

Übersicht

ID	Annahme	Begründung	Auswirkung	Datum	Autor
A01	Ein Modul wird immer im Kontext eines Studienprogramms angelegt und gehört dann fix zu diesem Programm.	Theoretisch könnte ein Modul zu mehreren Degree Programs gehören. Wurde aber vom Stakeholder als Anforderung im Forum spezifiziert.	Das UML-Klassendiagramm enthält eine 1:n-Beziehung zwischen Degree Program und Study Module.	25.05.2026	Daniel
A02	Problemstellung der „Calculate Function": Topologische Sortierung mit gerichtetem azyklischem Graphen.	Implementierung der Breitensuche als wahrscheinlich effizienteste Methode.	Aufbau des Codes im calculate unter ScheduleServiceImpl.	12.06.2026	Daniel
A03	Im StudyModuleRepository wird das Study Module unter einem Composed Key gespeichert: „Degree Program ID + # + Study Module Key".	Folge der Entscheidung, dass ein Modul nur zu einem Degree Program gehört. Das Modul „OOP" kann etwa in Computer Science und Electrical Engineering vorkommen; ohne Composed Key wäre es dupliziert.	Einfluss auf StudyModuleRepository.	12.06.2026	Daniel
A04	Es ist zulässig, Study Modules mit zyklischen Abhängigkeiten zu erstellen oder zu editieren.	Wir hätten eher dazu tendiert, die Prüfung beim Speichern/Updaten im Repository vorzunehmen. Die Anforderungen waren aber klar, dies im calculate zu implementieren; andernfalls wäre der Recalculate-Button hinfällig.	CyclicDependencyException muss nicht beim Erstellen/Update des Study Module abgefangen werden, sondern nur beim calculate von ScheduleServiceImpl.	16.06.2026	Daniel

A05	Wir nehmen an, dass jeder spezialisierte Controller eine spezielle Form eines allgemeinen Controllers darstellt.	Die Vererbung ist gerechtfertigt, weil zwischen den Klassen eine echte „Ist-eine"-Beziehung besteht. Ein StudyModuleController ist eine Art Controller, der speziell für Study Modules zuständig ist.	Damit ist die Voraussetzung für Vererbung erfüllt; wir definieren eine abstrakte Oberklasse.	16.06.2026	Daniel
A06	Model-Objekte (z. B. Module, DegreeProgram, Schedule) werden im jeweiligen Controller direkt erstellt und dem Service übergeben.	Direkte Erstellung im Controller wurde einer Mapping-Klasse vorgezogen, da vom Stakeholder als einfachste Lösung empfohlen und wir Einfachheit höher gewichten als zusätzliche Clean-Code-Struktur.	Mapping-Methoden im Controller müssen public sein (für Tests als Testee). Der Code ist leichter verständlich, jedoch weniger „clean".	19.06.2026	Daniel
A07	Die JSON-Serialisierung wird über Jackson-Mixin-Klassen in einem separaten Package umgesetzt.	Mixin-Klassen erlauben es, Jackson-Annotationen auszulagern, ohne die Model-Klassen zu „verschmutzen". Bei einem Wechsel der Serialisierungsmethode lassen sich die Mixin-Klassen flexibel anpassen.	Umsetzung im separaten Package (z. B. DegreeProgramMixin) plus JacksonConfig.	26.06.2026	Daniel
A08	Wir nehmen an, dass jedes spezialisierte Repository eine spezielle Form eines allgemeinen Repository darstellt.	Die Vererbung ist gerechtfertigt, weil zwischen den Klassen eine echte „Ist-eine"-Beziehung besteht. Ein JsonScheduleRepository ist ein Repository, das speziell für Schedules zuständig ist.	Damit ist die Voraussetzung für Vererbung erfüllt; wir definieren eine abstrakte Oberklasse.	26.06.2026	Daniel
A09	Fehlende Werte bei *Offered From* / *Offered To* (in der UI als „—" dargestellt) interpretieren wir als NULL-Einträge und werten sie als Datenqualitätsproblem: Der User hat das Modul unvollständig erfasst.	Theoretisch liesse sich NULL auch als offenes Intervall lesen — ein fehlendes *Offered To* würde dann „unbefristet angeboten" bedeuten. Wir haben dies intern diskutiert und uns dagegen entschieden.	Einfluss auf die Filter-Logik (Lambda/Stream) bei Study Modules: Ohne Filter werden NULL-Werte als *Undefined* behandelt und ans Listenende sortiert. Beim Filtern nach *Offered* dürfen sie nicht erscheinen.	10.07.2026	Daniel

A10	Logging erfolgt ausschliesslich über SLF4J (org.slf4j.Logger). Geloggt werden nur technische Fehler beim Lesen und Schreiben der JSON-Dateien (AbstractJsonRepository, Zeilen 14, 40, 51).	Erwartete Exceptions wie CyclicDependencyException (siehe A04) werden nicht geloggt: Sie entstehen durch Nutzereingabe, werden in der UI kommuniziert und vom Nutzer behoben. Log-Dateien bringen im aktuellen lokalen Betrieb keinen Mehrwert, wären aber bei einem späteren Server-Deployment wünschenswert (z. B. zur zentralen Log-Auswertung).	Technische Fehler werden von AbstractJsonRepository auf die Konsole geloggt, während fachlich erwartete Exceptions ausschliesslich als Flash-Message an den Nutzer kommuniziert werden und keinen Log-Eintrag hinterlassen. Bei einem späteren Server-Deployment wäre die Log-Ausgabe entsprechend zu konfigurieren.	10.07.2026	Daniel
A11	Bei den Modelklassen werden die Schedule- und Semesterklassen zu einem Record umgewandelt.	-Schedule und Semester können als record modelliert werden, weil es sich um einfache, unveränderliche Datenobjekte handelt. -DegreeProgram und StudyModule sollten nicht ohne Weiteres zu Records umgebaut werden, weil deren equals()/hashCode() bewusst auf Business Keys basieren und nicht auf allen Attributen. -Ein Java-record generiert equals() und hashCode() standardmässig über alle Record-Komponenten.	DegreeProgramm und StudyModule bleiben als Klassen erhalten.	10.07.2026	Marco

Details

Hintergrund zu A06: Es gab grundsätzlich drei Optionen, Model-Objekte zu erzeugen: (1) in der Service-Klasse durch Übergabe eines UI-Formulars (2) über eine separate Mapping-Klasse (3) direkt im Controller. Variante 2 wäre im Sinne von Clean Code die sauberste Lösung gewesen und war eine Empfehlung des Dozenten, hätte jedoch zusätzliche Komplexität durch eine weitere Klasse mit sich gebracht. Der Stakeholder empfahl auf Nachfrage ebenfalls Variante 3 als einfachste Lösung. Da Variante 2 und Variante 3 beide als valide gehalten, haben wir die Einfachheit höher gewichtet und uns dafür entschieden.